
Programmierung nach IEC 61131

Programmiersprachen IEC 61131

Textual: Instruction List (IL), Structured Text (ST)
Graphical: Function Block Diagram (FBD), Ladder Diagram (LD)

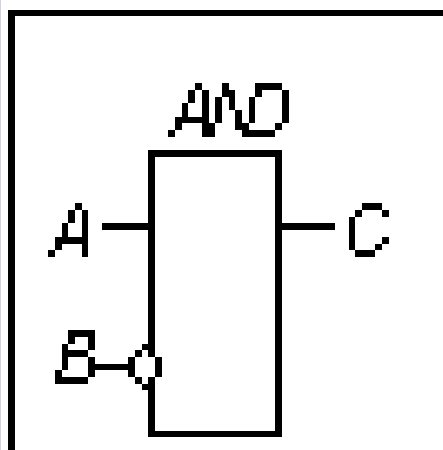
Instruction List (IL)

```
LD    A
ANDV  B
ST    C
```

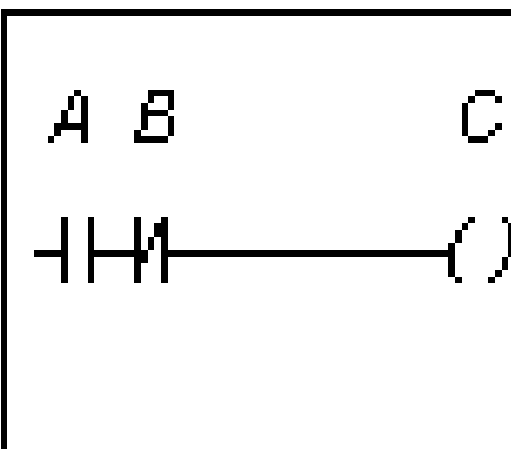
Structured Text (ST)

```
C = A AND NOT B
```

Function Block Diagram (FBD)

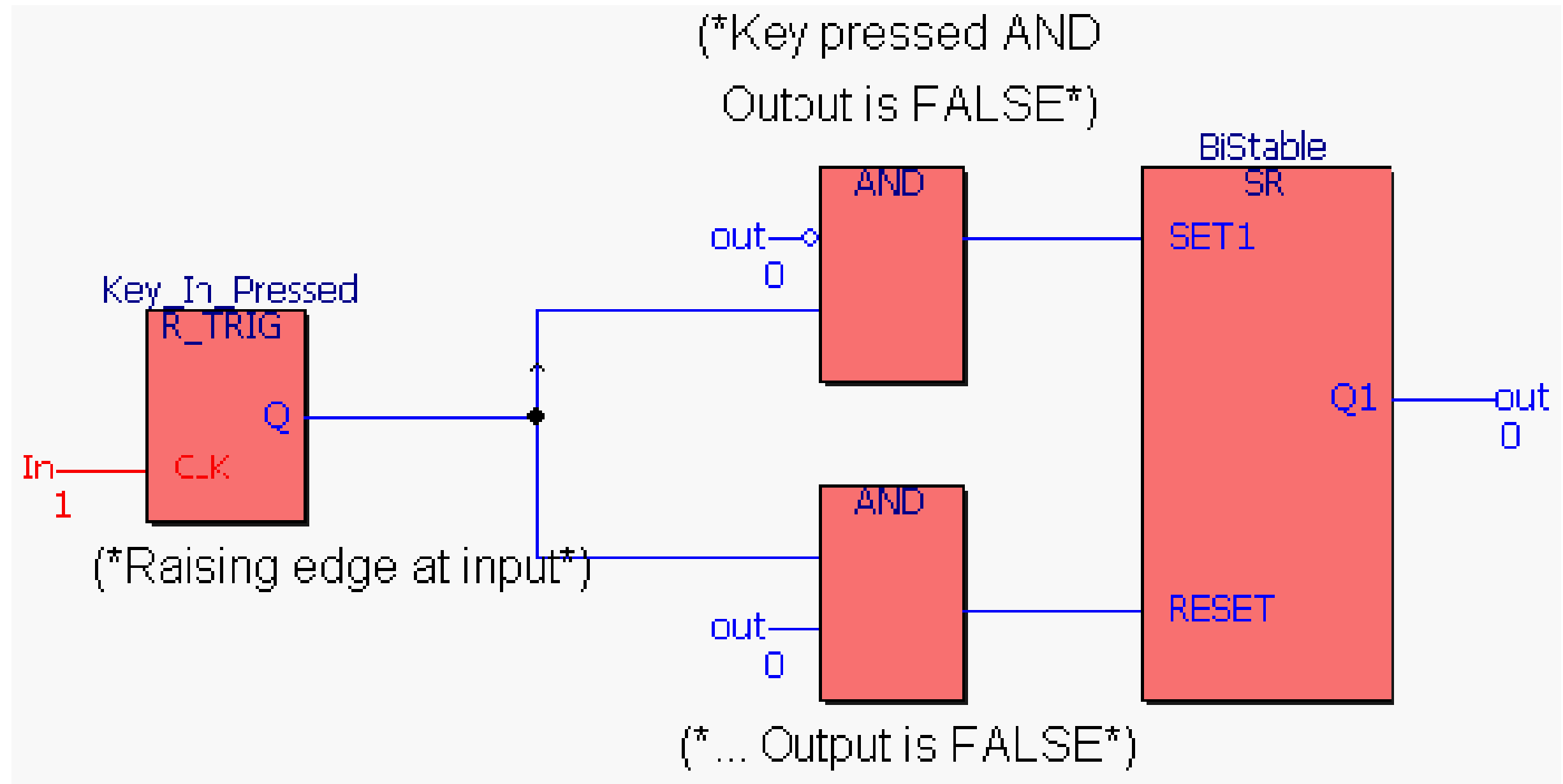


Ladder Diagram (LD)



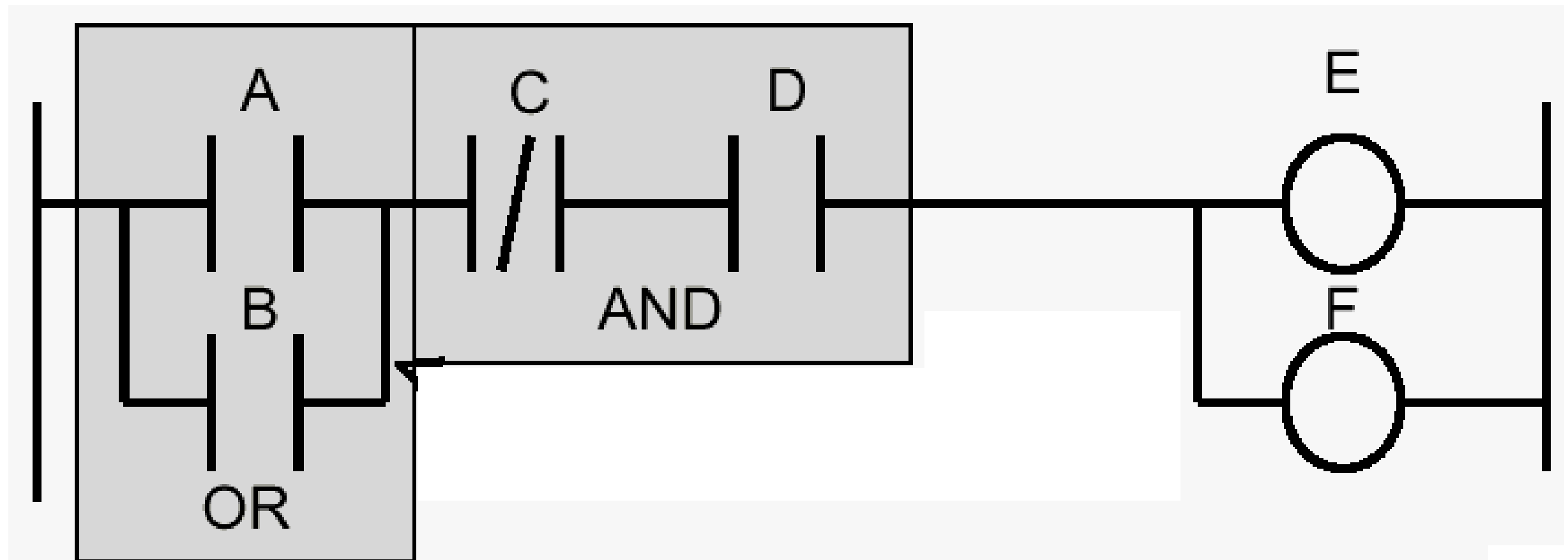
Programmiersprachen IEC 61131

Function Block Diagram (FBD)



Programmiersprachen IEC 61131

Ladder Diagram (LD)



IF ((A OR B) AND (NOT C) AND D) THEN E=1; F=1 END_IF

Programmiersprachen IEC 61131

Instruction List (IL)

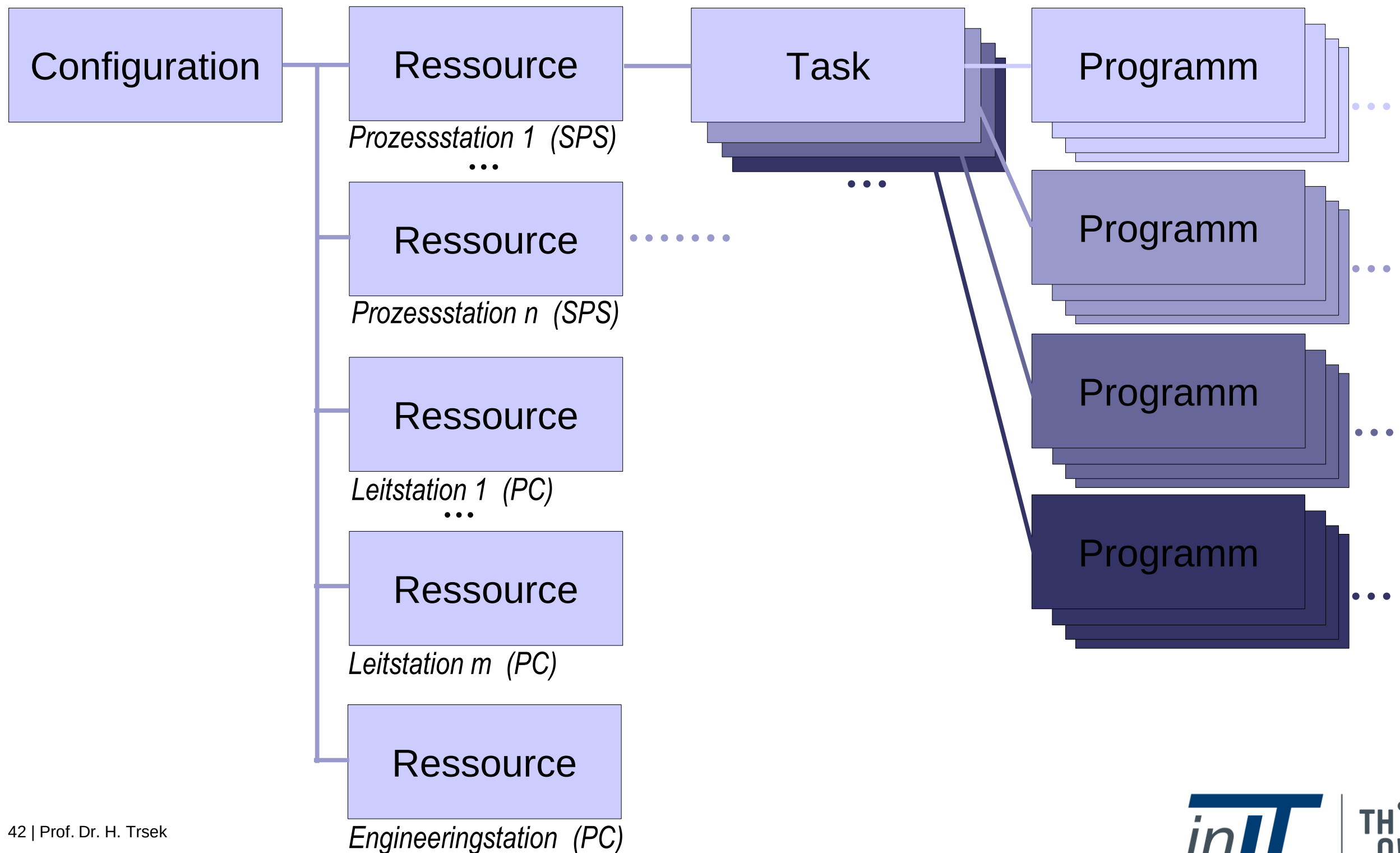
Label	Operator	Operand	Comment
START:	LD	%IX1	(* PUSH BUTTON *)
	ANDN	%MX5	(* NOT INHIBITED *)
	ST	%QX2	(* FAN ON *)

Programmiersprachen IEC 61131

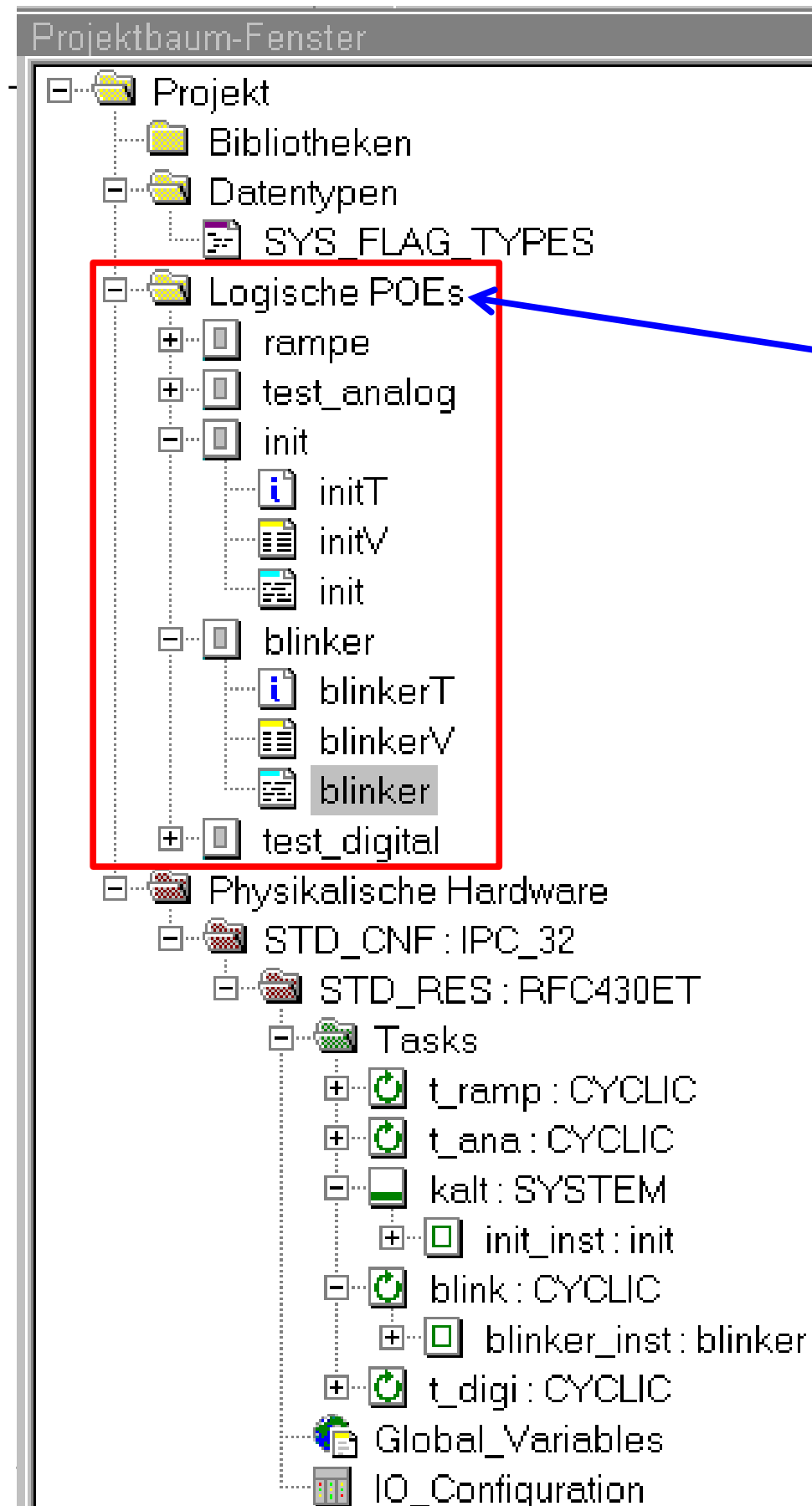
Structured Text (ST)

```
1  (* blinker *)
2  (* ----- *)
3
4  if x_blink_verriegelung = TRUE then
5      blinker_flag := blinker_flag XOR TRUE;
6      x_out_digi_0 := blinker_flag;
7      x_out_digi_1 := blinker_flag;
8      x_out_digi_2 := blinker_flag;
9      x_out_digi_3 := blinker_flag;
10     x_out_digi_4 := blinker_flag;
11     x_out_digi_5 := blinker_flag;
12     x_out_digi_6 := blinker_flag;
13     x_out_digi_7 := blinker_flag;
14 end_if;
```

Projektbaum IEC 61131

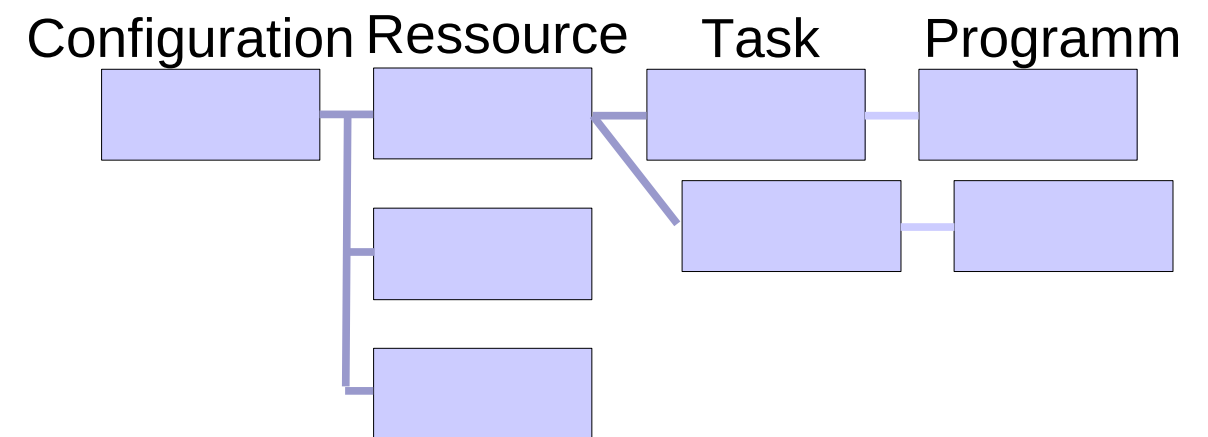
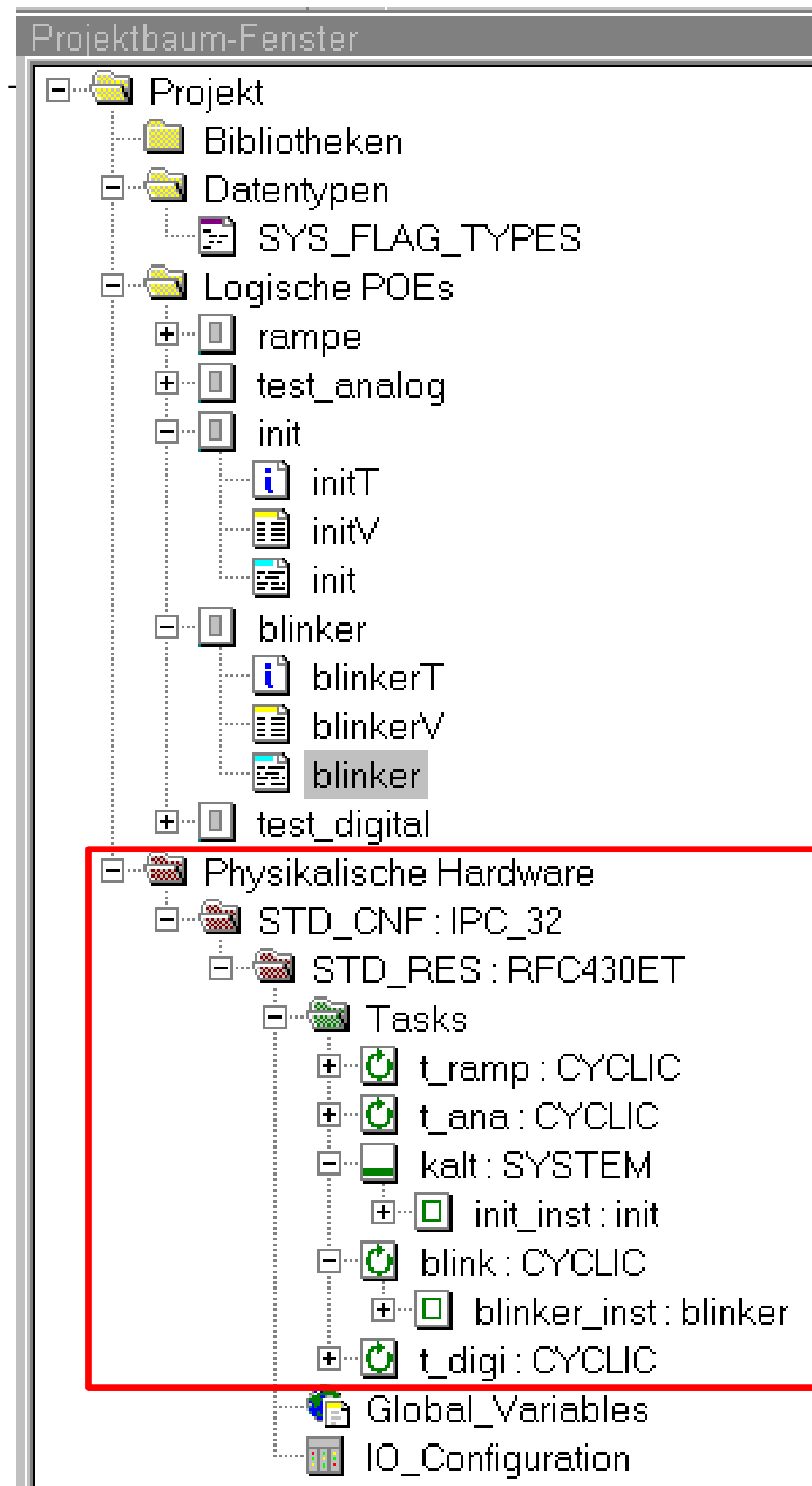


Projektbaum PC WORX



Programmorganisationseinheit

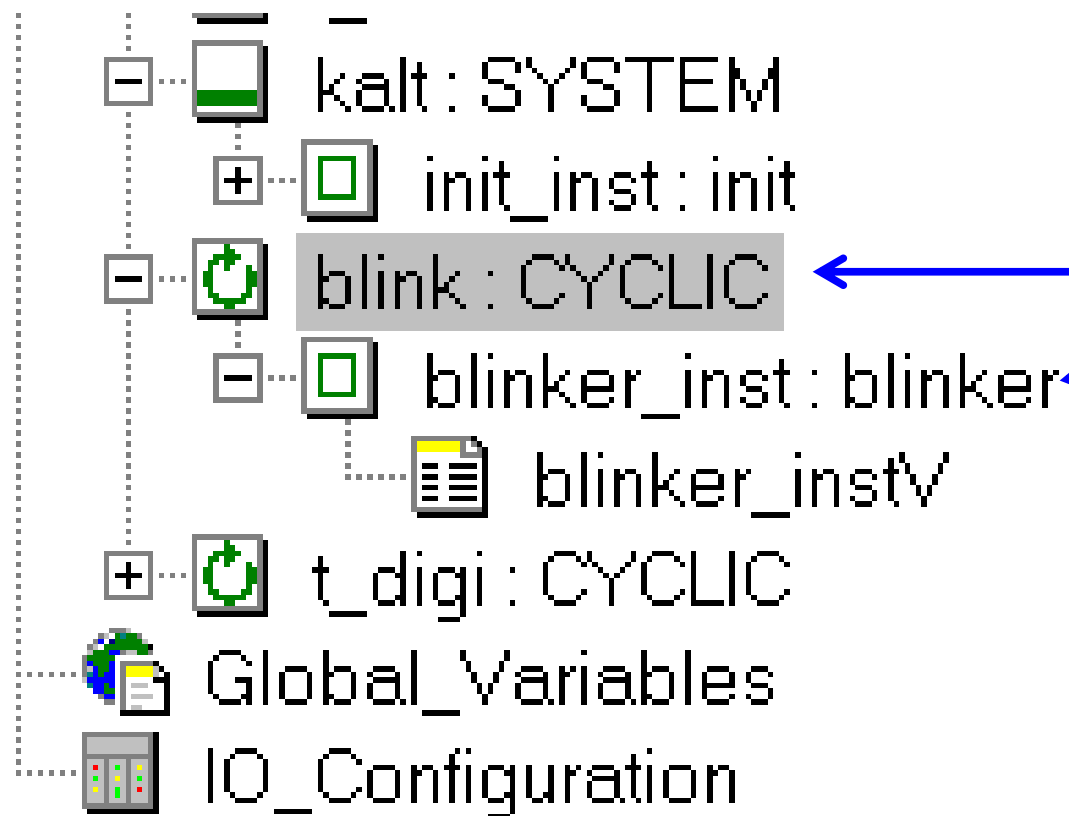
Projektbaum PC WORX



CONFIGURATION
RESSOURCE

Task
Programminstanz

Projektbaum PC WORX



Zyklisch eingeplante Task

Task **blink**

Programm **blinker_inst**
(Instanz der logischen
POE **blinker**)

Einplanungsintervall 1 Sekunde
(Task-Einstellungen für Task **blink**)

Task-Einstellungen für IPC_32

Intervall:	1000	ms
Priorität:	0	
Watchdogzeit:	100	ms

Projektbaum PC WORX

Prozessdatenzuordnung

Symbole/Variablen

- [-] STD_CNF : ARM_L_40
 - [-] STD_RES : ILC350ET
 - Default
 - System Variables
 - [-] ez : CYCLIC
 - + ez1_inst : ez1

- [-] UNTITLED
 - [-] ILC 350 ETH 193.16.113.38
 - [-] Ressource
 - STD_RES ILC350ET
 - [-] # INTERBUS 0 . 0
 - # 1 IB IL 24 DO 8 0 . 1
 - # 2 IB IL 24 DI 8 0 . 2
 - # 3 IB IL AO 1/SF 0 . 3
 - # 4 IB IL AI 2/SF 0 . 4
 - Nicht verbunden

Symbol/...	Datentyp	Prozessdatum	Beschreibung
schalter0	BOOL		
led0	BOOL	# 1 IB IL 24 DO 8 ...	

Gerät	Prozessdatum	I/Q	Datentyp
# 1 I...	1.1.1	Q	BOOL
# 1 I...	1.2.1	Q	BOOL
# 1 I...	2.1.1	Q	BOOL
# 1 I...	2.2.1	Q	BOOL
# 1 I...	3.1.1	Q	BOOL
# 1 I...	3.2.1	Q	BOOL
# 1 I...	4.1.1	Q	BOOL
# 1 I...	4.2.1	Q	BOOL
# 1 I...	~DO 8	Q	BYTE

Instruction List (IL), Anweisungsliste (AWL)

Anweisungen IL (Auswahl)

LD	Lade Operand nach Akku
ST	Speichere Akku nach Operand
LDN	Lade Operand negiert nach Akku
STN	Speichere Akku negiert nach Operand
AND	Akku UND Operand, Ergebnis nach Akku
ANDN	Akku UND Operand negiert, Ergebnis nach Akku
OR	Akku ODER Operand, Ergebnis nach Akku
ORN	Akku ODER Operand negiert, Ergebnis nach Akku
XOR	Akku EXOR Operand, Ergebnis nach Akku
XORN	Akku EXOR Operand negiert, Ergebnis nach Akku
EQ	Wenn Akku gleich Operand, Logisch 1 nach Akku, sonst logisch 0
NE	Wenn Akku ungleich Operand, Logisch 1 nach Akku, sonst logisch 0
GT	Wenn Akku größer Operand, Logisch 1 nach Akku, sonst logisch 0
GE	Wenn Akku größer gleich Operand, Logisch 1 nach Akku, sonst logisch 0
LT	Wenn Akku kleiner Operand, Logisch 1 nach Akku, sonst logisch 0
LE	Wenn Akku kleiner gleich Operand, Logisch 1 nach Akku, sonst logisch 0

Instruction List (IL), Anweisungsliste (AWL)

Anweisungen IL (Auswahl)

ADD Akku plus Operand nach Akku

SUB Akku minus Operand nach Akku

MUL Akku mal Operand nach Akku

DIV Akku geteilt durch Operand nach Akku

JMP Springe zur Marke, die im Operandenfeld angegeben ist; unbedingt

JMPC Springe, wenn Akku = logisch 1

JMPCN Springe, wenn Akku = logisch 0

Structured Text (ST), Strukturierter Text (ST)

Sprachelemente ST (Auswahl)

Beispiel: `led1 := schalter1 AND NOT schalter2 ;`

Operatoren:

<code>:=</code>	Zuweisung	<code>></code>	größer	<code>NOT</code>	Komplement
<code>+</code>	Addition	<code>>=</code>	größer gleich	<code>AND</code>	bitweises UND
<code>-</code>	Subtraktion	<code><</code>	kleiner	<code>OR</code>	bitweises ODER
<code>*</code>	Multiplikation	<code><=</code>	kleiner gleich	<code>XOR</code>	bitweises Exklusiv-ODER
<code>/</code>	Division	<code>=</code>	gleich		
<code>**</code>	Exponent	<code><></code>	ungleich		

Structured Text (ST), Strukturierter Text (ST)

Sprachelemente ST (Auswahl)

```
if schalter5 = TRUE then
    led0 := TRUE;
    led1 := FALSE;
    analog := WORD#16#03FC;
else
    led7 := TRUE;
    led8 := TRUE;
end_if;
```

```
if counter <= 25 then
    venti18 := TRUE;
end_if;
```

```
case state of
    0:    if ...
          ...
    1:    ...
    2:    ...
          ...
        else
          ...
end_case;
```


Vergleich IL und ST am Beispiel Blinken

Einplanungszeit der Task : 1000 ms

IL

```
ld    freigabe  
jmpcn ende
```

```
ld    blink  
stn   blink
```

```
st    led0  
st    led1
```

```
...  
ende:
```

ST

```
if freigabe = true then  
    blink := not blink;
```

```
    led0 := blink;  
    led1 := blink;
```

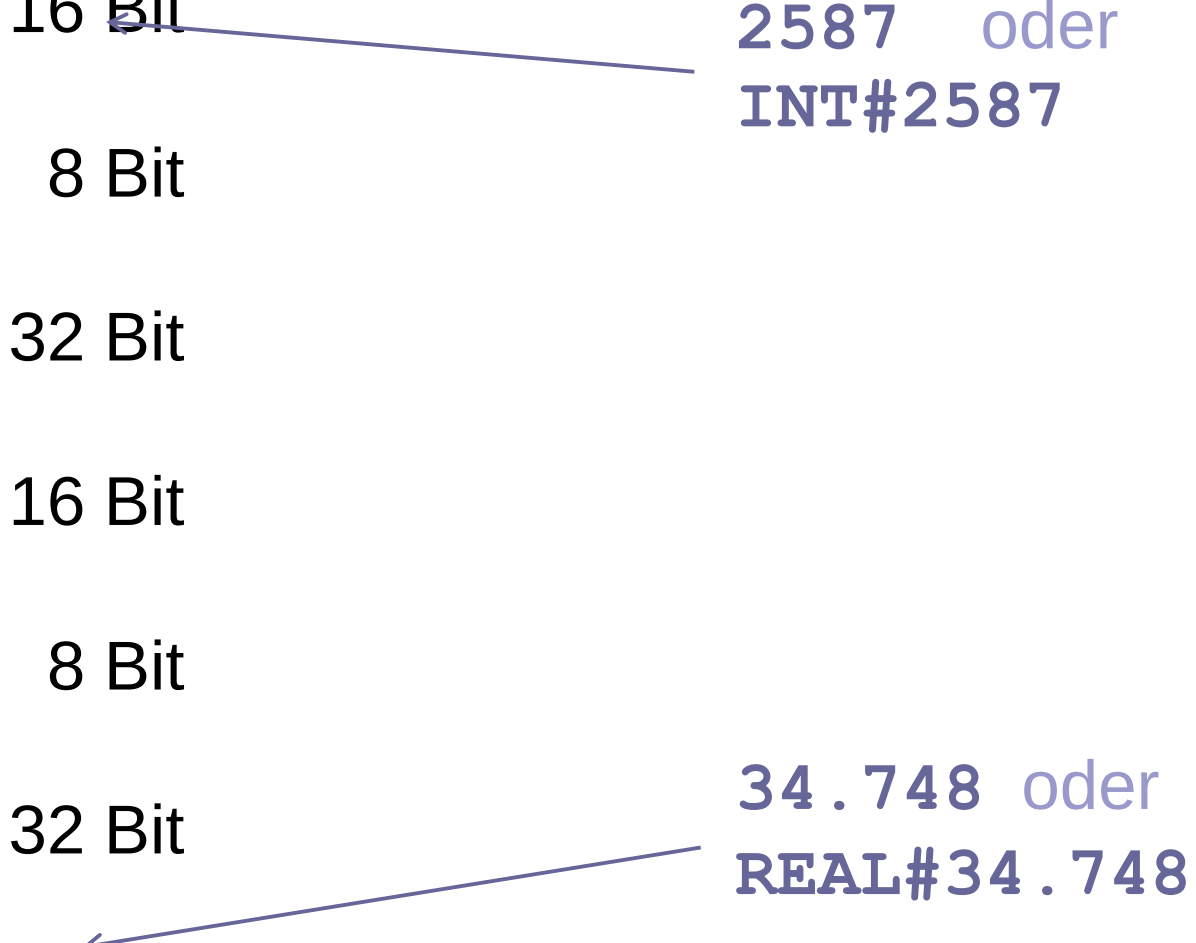
```
    ...  
end_if;
```


Structured Text (ST)

Datentypen

Darstellung als Konstanten
Beispiele:

INT	16 Bit	2587 oder INT#2587
SINT	8 Bit	
DINT	32 Bit	
UINT	16 Bit	
USINT	8 Bit	
UDINT	32 Bit	34.748 oder REAL#34.748
REAL	32 Bit	



Structured Text (ST)

Datentypen

Darstellung als Konstanten

BOOL

1 Bit

TRUE, FALSE oder **BOOL#TRUE, BOOL#FALSE**

BYTE

8 Bit

BYTE#15 oder **BYTE#16#0F** oder **BYTE#2#0000_1111**

WORD

16 Bit

WORD#7514 oder **WORD#16#1D5A** oder
WORD#2#0001_1101_0101_1010

DWORD

32 Bit

...